

BotShield Browser Extension

A Manifest V3 extension that scans commenters on the page you're viewing, calls your BotShield API, and overlays a warning badge next to likely-bot accounts.

Folder layout — create a folder `botshield-extension/` with these 4 files, then load it via `chrome://extensions` → **Load unpacked**.

1. `manifest.json`

```
{
  "manifest_version": 3,
  "name": "BotShield",
  "version": "0.1.0",
  "description": "Warns you when a chat participant or commenter is likely a bot or fake account.",
  "permissions": ["storage", "activeTab"],
  "host_permissions": ["https://api.yourdomain.com/*"],
  "background": { "service_worker": "background.js" },
  "content_scripts": [
    {
      "matches": ["https://www.youtube.com/*", "https://www.reddit.com/*"],
      "js": ["content.js"],
      "run_at": "document_idle"
    }
  ],
  "action": { "popup": "popup.html" }
}
```

2. `content.js` — finds commenters, calls API, injects badges

```
const API = "https://api.yourdomain.com/assess";
// Per-platform selectors: [handle element, container to badge]
const SELECTORS = {
  "youtube.com": {
    author: "yt-formatted-string#author-text, a.yt-simple-endpoint.yt-formatted-string",
    anchor: "ytd-comment-thread-renderer",
  },
  "reddit.com": {
    author: "a[data-testid='post_author_link'], a[data-testid='comment_author_link']",
    anchor: "[data-testid='comment'], shreddit-post",
  },
};
const verdictStyles = {
```

```

HUMAN_LIKELY: { color: "#16a34a", icon: "✓", text: "Human-likely" },
LOW: { color: "#16a34a", icon: "✓", text: "Human-likely" },
MEDIUM: { color: "#f59e0b", icon: "▲", text: "Caution – bot-like signals" },
HIGH: { color: "#ea580c", icon: "🤖", text: "Very likely automated – avoid
engaging" },
BOT: { color: "#dc2626", icon: "🤖", text: "Bot – do not engage, report" },
};

async function assessHandle(handle) {
  const res = await fetch(`${API}?handle=${encodeURIComponent(handle)}`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(collectSignals(handle)),
  });
  return res.json(); // { label, score, signals: [...] }
}

// Collect on-page signals for the account (publicly visible only)
function collectSignals(handle) {
  return {
    handle,
    comments: getCommentsBy(handle),
    profileHints: getProfileHints(handle),
  };
}

function getCommentsBy(handle) {
  // Return [{text, postedAtISO}] for this handle – adapt per platform DOM
  return [...document.querySelectorAll(commentTextSelector())]
    .filter(el => el.closest(selectors().anchor)?.innerText.includes(handle))
    .slice(0, 20)
    .map(el => ({ text: el.innerText, postedAtISO: new Date().toISOString() }));
}

function injectBadge(anchorEl, verdict) {
  if (anchorEl.querySelector(".botshield-badge")) return;
  const style = verdictStyles[verdict.label] ?? verdictStyles.LOW;
  const badge = document.createElement("span");
  badge.className = "botshield-badge";
  badge.title = (verdict.signals || []).join("\n");
  Object.assign(badge.style, {
    color: "white", backgroundColor: style.color,
    fontSize: "11px", padding: "1px 6px",
    borderRadius: "10px", marginLeft: "6px",
    cursor: "help", fontWeight: "600",
  });
  badge.textContent = `${style.icon} ${style.text}`;
  anchorEl.querySelector(selectors().author)?.appendChild(badge);
}

// --- Orchestration with caching ---
const cache = new Map();
async function scan() {
  const anchors = [...document.querySelectorAll(selectors().anchor)];
  const handles = [...new Set(
    anchors.map(a =>
a.querySelector(selectors().author)?.innerText.trim()).filter(Boolean)
  )];
  for (const handle of handles) {

```

```

if (cache.has(handle)) continue;
cache.set(handle, null); // in-flight guard
try {
const verdict = await assessHandle(handle);
cache.set(handle, verdict);
const anchor = anchors.find(a =>
a.querySelector(selectors().author)?.innerText.trim() === handle);
if (anchor) injectBadge(anchor, verdict);
} catch (e) { cache.delete(handle); }
}
}
let scheduled = null;
const observer = new MutationObserver(() => {
clearTimeout(scheduled);
scheduled = setTimeout(scan, 800); // debounce: YouTube mutates DOM constantly
});
observer.observe(document.body, { childList: true, subtree: true });
scan();
function selectors() { return SELECTORS[location.hostname.replace("www.", "")] ?? {}; }
function commentTextSelector() {
return location.hostname.includes("youtube.com")
? "yt-formatted-string#content-text"
: "[data-testid='comment'] p, .md p";
}
function getProfileHints() { return {}; } // extension v2: hover-fetch channel page

```

3. **background.js** — quota + rate limiting

```

const WINDOW_MS = 60_000, MAX_PER_WINDOW = 30;
let timestamps = [];
chrome.runtime.onMessage.addListener((msg, _sender, sendResponse) => {
if (msg.type !== "assess") return;
const now = Date.now();
timestamps = timestamps.filter(t => now - t < WINDOW_MS);
if (timestamps.length >= MAX_PER_WINDOW) {
sendResponse({ error: "rate_limited" });
return;
}
timestamps.push(now);
fetch(`https://api.yourdomain.com/assess`, {
method: "POST",
headers: { "Content-Type": "application/json" },
body: JSON.stringify(msg.payload),
})
.then(r => r.json())
.then(sendResponse);
return true; // async response
});

```

4. `popup.html` — settings

```
<!DOCTYPE html>
<html>
<body style="width:260px;font-family:system-ui">
  <h3>🛡 BotShield</h3>
  <label><input type="checkbox" id="hideBots"> Hide HIGH/BOT comments
entirely</label>
  <label><input type="checkbox" id="onlyBadge"> Only badge, never hide
(default)</label>
  <p style="font-size:11px;color:#666">Verdicts are probabilities, not accusations.
False positives happen – report mistakes to improve the model.</p>
  <script>

for (const id of ["hideBots", "onlyBadge"])

document.getElementById(id).onchange = e =>

chrome.storage.sync.set({ [id]: e.target.checked });

</script>
</body>
</html>
```

Legal & privacy notes (built into the design)

- Reads only what the user already sees on-screen — no background scraping, no bulk collection.
- Sends to your API only: handle, comment texts, timestamps. No other page data.
- Add a privacy policy covering this processing (GDPR Art. 6(1)(f)/13).
- Badge copy uses "likely/very likely" language — keep it probabilistic.